

**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

Declaración de Diseño del Frontend

MalphasOS
Gestión de Clientes

Sean Sebastián Bolívar Calderón - 20231020135

13 de septiembre de 2026

Contents

Nota preliminar	3
1. Alcance	3
1.1. Qué cubre	3
1.2. Qué queda fuera, y por qué	3
2. Lo que la especificación impone	4
2.1. Los seis requisitos de interfaz	4
2.2. Los umbrales de tiempo	4
2.3. El trabajo en campo	5
2.4. Los requisitos que sólo esperan un formulario	5
3. Decisiones de plataforma	5
3.1. Framework: Angular	5
3.2. Estilos: Tailwind con la paleta oficial como tokens	6
3.3. Componentes: spartan/ui sobre Angular CDK	6
3.4. Datos: TanStack Query, aislado tras un servicio por módulo	7
3.5. Formularios: los reactivos de Angular	7
3.6. Contrato: el cliente se genera desde OpenAPI	7
4. Arquitectura	7
4.1. Organización por módulo de negocio	7
4.2. Capas dentro de un módulo	8
4.3. Autenticación y autorización	8
4.4. Traducción de errores	9
5. Sistema visual	9
5.1. La fuente es el manual de marca	9
5.2. Color	9
5.2.1. La regla de contraste del manual, y el caso que no nombra	10
5.3. Tipografía, retícula y espaciado	10
5.4. Densidad táctil y orden de diseño	11
5.5. Tono de los textos de la interfaz	11
5.6. Cómo se cuenta un paso (RNF-15)	11

6. Accesibilidad	11
7. Movilidad y comportamiento sin conexión	12
7.1. Lo que se construye ahora	12
7.2. Por qué aplazarlo no incumple la especificación	12
7.3. Lo que se añadirá después, y con qué forma	13
7.4. Por qué la escritura sin conexión no entrará sin tocar el backend	13
7.5. Decisiones ya tomadas para cuando se retome	13
8. Estrategia de pruebas	14
9. Plan de construcción	14
10. Riesgos asumidos y lo que este documento no decide	14
11. Trazabilidad	15

Nota preliminar

Qué es este documento. La declaración de las decisiones de diseño del frontend de MalphasOS: plataforma, arquitectura, sistema visual, accesibilidad, comportamiento sin conexión y estrategia de pruebas. Cada decisión va con su porqué y con su coste, porque una decisión sin coste escrito se relee más tarde como si no lo hubiera tenido.

Qué no es. No es un manual de estilo ni una guía de componentes: eso se escribirá cuando los componentes existan. Tampoco es una lista de pantallas. Y no sustituye a la ERS: **este documento no introduce ningún requisito nuevo**, sólo decide cómo se satisfacen los que ya están escritos.

Por qué existe y por qué ahora. El backend está terminado en sus cinco módulos y no hay una sola línea de frontend. Esa asimetría tiene un efecto medible: de los 31 requisitos funcionales de la ERS hay 11 implementados, y **cuatro de los que faltan en órdenes de trabajo están abiertos únicamente porque no existe el formulario**, no porque falte nada en el servidor. Seis requisitos no funcionales enteros —RNF-12 a RNF-17— no tienen relación alguna con el backend. Un backend completo con cero frontend cumple cero de ellos.

De dónde salen las restricciones. De la ERS, no de la preferencia de quien escribe. El apartado 2.1 exige que el sistema sea accesible desde el teléfono del ingeniero **sin instalar aplicaciones nativas**; el 3.1.2 lista la pantalla táctil entre las interfaces de hardware; el 3.1.1 recoge los seis requisitos de interfaz. Donde este documento decide algo que la ERS no dice, se señala explícitamente como decisión propia.

1. Alcance

1.1. Qué cubre

La **aplicación operativa** de MalphasOS: la interfaz con la que los administradores gestionan clientes, equipos y órdenes de trabajo, y con la que los ingenieros consultan y ejecutan su trabajo en campo. Cubre las decisiones transversales que afectan a todas sus pantallas.

1.2. Qué queda fuera, y por qué

La *landing page* de la empresa queda fuera de esta declaración. El cronograma del documento de constitución la sitúa en la semana 1, de modo que su ausencia aquí podría leerse como un olvido. No lo es:

- **Es otro producto.** Es pública, no tiene autenticación, no consume el API y su objetivo es comercial, no operativo.
- **No avanza ninguno de los seis requisitos de interfaz.** RNF-12 a RNF-17 hablan de la aplicación que un ingeniero usa, no de una página de presentación.
- **Pide un stack distinto.** Una página pública necesita indexación por buscadores, y una aplicación de página única es precisamente la peor forma de conseguirla. Construir la dentro de la aplicación operativa la perjudicaría.

Se decidirá por separado cuando la aplicación esté en pie. Lo único que este documento

fija sobre ella es que **heredará la paleta oficial**, para que los dos productos se reconozcan como de la misma casa.

Queda fuera del primer alcance, pero decidido, el comportamiento sin conexión y la instalación en el dispositivo. No se descartan: la aplicación los tendrá. Se aplazan, y la sección 7 deja escrito con qué forma entrarán y por qué aplazarlos no incumple ningún requisito.

También queda fuera todo lo que dependa de módulos de backend que no existen: reportes de servicio, firma digital, hojas de vida, inventario, alertas y módulo comercial. Sus pantallas se diseñarán cuando exista contra qué construirlas.

2. Lo que la especificación impone

Esta sección no decide nada. Recoge lo que ya está escrito y que el diseño tiene que satisfacer, para que las decisiones de las secciones siguientes se lean contra algo y no en el vacío.

2.1. Los seis requisitos de interfaz

Código	Qué pide	Qué obliga a decidir
RNF-12	Accesibilidad web	Un nivel concreto; la ERS no fija ninguno
RNF-13	Diseño responsivo	Un orden: móvil primero o escritorio primero
RNF-14	Interacción por pantalla táctil	Un tamaño mínimo de área táctil
RNF-15	Procesos en ≤ 6 pasos	Cómo se cuenta un paso
RNF-16	Listas, autocompletado y selección múltiple	De dónde salen esos componentes
RNF-17	Mensajes de error claros	Quién traduce los códigos del backend

RNF-17 merece una precisión, porque es el único de los seis que tiene ya media solución construida: el backend emite códigos estables por módulo (`ERR_WORK_ORDER_001` y sus hermanos), con la regla explícita de que un código de “no existe” nunca se comparte con uno de “datos inválidos”. Lo que falta no es el catálogo sino **quien lo traduzca a una frase que una persona lea**. Esa pieza es responsabilidad del frontend y de nadie más.

2.2. Los umbrales de tiempo

Umbral	Dónde aparece
2 segundos	RNF-03, RNF-04, RNF-09, RNF-10, RNF-18, RNF-22 — respuesta a consultas
3 segundos	Apartado 3.3 de la ERS, OBJ-07 y CE-08 del documento de constitución
3 minutos	RNF-01 y RNF-20 — procesos completos de usuario

Los dos primeros son compatibles: dos segundos satisfacen un umbral de tres. **Ningún documento lo dice**, y esta lectura es de quien contrasta, no un hecho declarado. Este

documento adopta **el umbral más exigente, dos segundos**, porque satisface a los dos y evita tener que reconciliarlos.

2.3. El trabajo en campo

Tres apartados de la ERS describen el escenario real de uso, y juntos son la restricción de diseño más fuerte del documento:

- **Apartado 1.2:** el sistema “facilitará el trabajo en campo de los ingenieros mediante el uso de dispositivos móviles”.
- **Apartado 2.1:** accesible desde el teléfono del ingeniero **sin instalar aplicaciones nativas**.
- **Apartado 3.1.2:** la pantalla táctil figura entre las interfaces de hardware del sistema, junto a la cámara.

La consecuencia es que **el caso principal no es un escritorio**. Un ingeniero de pie en el pasillo de un hospital, con una mano ocupada, es el usuario que el diseño debe servir primero.

2.4. Los requisitos que sólo esperan un formulario

Cuatro requisitos de órdenes de trabajo están abiertos con el backend completo:

Código	Qué pide	Qué falta
RF-03	Formulario de orden: cliente, sede, áreas, tipo y periodicidad	Los cinco datos existen en el API; el formulario no
RF-04	Selección múltiple de áreas de la sede	Es un paso de interfaz. Las áreas elegidas no se persisten a propósito : quedan implícitas en los equipos
RF-06	Visualización de equipos por área	La consulta existe; falta la interfaz que agrupe
RF-07	Selección múltiple de equipos	El API añade de uno en uno; la selección múltiple es del formulario

Cerrar estos cuatro es el objetivo del primer bloque de construcción (sección 9).

3. Decisiones de plataforma

3.1. Framework: Angular

La aplicación se construye con Angular.

Por qué. Angular impone inyección de dependencias y una estructura de módulos y servicios que **es el espejo natural de la arquitectura hexagonal del backend**. Un servicio inyectable que encapsula las llamadas de un módulo es lo que en el backend es un puerto; un guard de ruta es lo que allí es una anotación de autorización. Esa correspondencia hace que las dos mitades del sistema se puedan razonar con el mismo vocabulario, lo

que importa en un proyecto cuya documentación insiste en que las decisiones se comparen entre capas.

Qué cuesta, y queda escrito. El proyecto original (`bolivarbioingenieria-app`) tiene un starter de autenticación en `Frontend/src/auth/` —instancia de Keycloak, proveedor de contexto, guard de ruta y ayudante de peticiones— que la documentación técnica clasificó como **directamente reutilizable**. Elegir Angular lo descarta: es React. Hay que reescribirlo contra `keycloak-angular`.

Son cuatro piezas pequeñas y el patrón se conserva, pero **es la primera vez que este proyecto desecha algo marcado como reutilizable**, y se hace con los ojos abiertos: la coherencia estructural con el backend se consideró más valiosa que ahorrar la reescritura de cuatro archivos.

Versión. **No se fija aquí una versión de memoria.** Se adopta la estable en el momento de crear el proyecto y **se anota en este mismo apartado al crearlo**, igual que el backend fija Spring Boot 4.1.1. Una versión escrita de antemano y nunca comprobada es una afirmación que nace falsa.

3.2. Estilos: Tailwind con la paleta oficial como tokens

Tailwind CSS, con los colores oficiales del proyecto declarados como tokens del tema.

La paleta no se inventa: ya existe y es la de la ERS, el documento de constitución y la matriz de trazabilidad. Reutilizarla es lo que hace que la aplicación y sus documentos se reconozcan como del mismo producto. El detalle del sistema visual va en la sección 5, incluido **un conflicto real entre esa paleta y el nivel de accesibilidad que este documento declara**.

3.3. Componentes: spartan/ui sobre Angular CDK

Los componentes de interfaz se toman de spartan/ui.

El planteamiento importa más que la librería: **el código de cada componente se copia al repositorio** en vez de consumirse como caja negra, y debajo van las primitivas del Angular CDK, que resuelven lo difícil de verdad —superposiciones, atrapado de foco, navegación de listas por teclado, anuncios para lectores de pantalla—.

Por qué no una librería cerrada. RNF-14 exige uso táctil y RNF-12 accesibilidad verificable. Las dos cosas obligan a ajustar áreas de pulsación y comportamiento de foco componente a componente. Con el código en el repositorio eso es una edición; con una librería cerrada es una pelea contra su sistema de temas.

Qué cubre de RNF-16. Los tres componentes que el requisito nombra —listas, autocompletado y selección múltiple— existen como primitivas accesibles y no hay que escribirlos desde cero.

Lo que hay que rehacer, y es trabajo real. Las plantillas de `spartan/ui` traen esquinas redondeadas y una escala de color propias. **El manual de marca exige radio cero** y su propia paleta, de modo que cada componente que se incorpore se re-viste antes de usarse. Que el código viva en el repositorio es precisamente lo que hace eso posible: con una librería cerrada habría que pelear contra su sistema de temas para conseguir lo

mismo.

3.4. Datos: TanStack Query, aislado tras un servicio por módulo

El estado de servidor se gestiona con TanStack Query.

Resuelve caché, deduplicación de peticiones, reintentos, estados de carga y error, e invalidación después de escribir. Eso último es lo que impide el defecto clásico de dos pantallas que muestran la misma lista y sólo una se entera del cambio.

El riesgo, declarado. El adaptador de Angular se distribuye con el sufijo *experimental* en el nombre del paquete. Es una dependencia inestable en el centro de la capa de datos y no se oculta.

Cómo se acota. Ningún componente usa TanStack Query directamente. Cada módulo expone un servicio inyectable propio —`WorkOrderApiService` y sus hermanos— y la librería vive dentro de él. Si su API cambia entre versiones, lo que se toca son esos servicios y no cada pantalla. Es la misma idea por la que el backend publica puertos en vez de dejar que un módulo llame al almacén de otro.

3.5. Formularios: los reactivos de Angular

Formularios reactivos, con validación declarada en el modelo.

La razón no es de gusto sino de requisito: **RNF-07 pide impedir editar manualmente los datos autocompletados.** Eso no es un atributo visual de “sólo lectura” sino una regla del formulario, y un formulario reactivo la puede expresar y probar sin montar la pantalla.

3.6. Contrato: el cliente se genera desde OpenAPI

Los tipos y los clientes de llamada se generan a partir de los documentos OpenAPI que el backend publica, y no se escriben a mano.

El backend declara un grupo por módulo y **tres de ellos ya tienen prueba que verifica que el grupo contiene sus recursos.** Generar desde ahí extiende esa garantía al frontend: si el backend renombra un campo, el frontend **deja de compilar.** Escribir los tipos a mano convierte ese mismo cambio en un fallo en tiempo de ejecución, o en ninguno.

Lo generado se versiona en el repositorio. Así el proyecto compila sin necesidad de un backend levantado, y los cambios del contrato aparecen como diferencias revisables en lugar de como un resultado distinto del generador.

4. Arquitectura

4.1. Organización por módulo de negocio

El código se organiza por módulo de negocio, con los mismos nombres que el backend, y no por tipo técnico.

`src/app/`

core/	sesion, interceptores, traduccion de errores	
shared/	componentes de interfaz, utilidades	
features/		
client/	equipment/	person/
location/	workOrder/	

Por qué no por tipo técnico. El frontend del proyecto original está organizado en `pages/`, `services/` y `auth/`, y la documentación técnica dice explícitamente que **no hay ahí una convención que copiar** y que corresponde a MalphasOS decidirla. Repetir esa estructura reparte cada módulo entre tres carpetas y obliga a saltar entre ellas para leer una función completa.

Qué gana espejar los nombres. La matriz de trazabilidad relaciona requisitos con código. Si `features/workOrder/` se llama igual que el paquete del backend, la relación se lee sin traducción.

4.2. Capas dentro de un módulo

Capa	Responsabilidad
Cliente generado	Tipos y llamadas, producidos desde OpenAPI. No se edita a mano
Servicio del módulo	Encapsula TanStack Query y el cliente generado. Es la única puerta al servidor
Componentes	Presentación e interacción. No conocen la librería de datos ni el transporte
Modelo de vista	Traduce entre lo que el API devuelve y lo que la pantalla necesita, cuando no coinciden

La regla que sostiene el conjunto: **un componente nunca llama al API directamente**. Siempre pasa por el servicio de su módulo.

4.3. Autenticación y autorización

Autenticación con `keycloak-angular` sobre el flujo de código de autorización con PKCE, que es el correcto para un cliente público. El patrón del proyecto original se conserva aunque su código no: instancia única, inicio de sesión al arrancar, refresco automático del token antes de que expire y cierre de sesión si el refresco falla.

Autorización. El backend define 19 autoridades y exige una sola, nombrada literalmente, en cada una de sus 92 operaciones. El frontend **refleja esas mismas autoridades en los guards de ruta y en la visibilidad de los controles**.

Dos precisiones que evitan malentendidos peligrosos:

- **El frontend no autoriza: oculta.** Esconder un botón mejora la experiencia; el permiso lo sigue comprobando el servidor en cada llamada. Un frontend que “protege” una operación no protege nada.
- **El vocabulario no se duplica a mano.** Las autoridades salen del token, y la lista se contrasta con la del backend. Dos listas escritas por separado se desincronizan, y ésta ya tiene un precedente documentado en el propio proyecto.

4.4. Traducción de errores

Un catálogo por módulo traduce cada código del backend a un mensaje en español, con un mensaje genérico de respaldo para los códigos que aún no tenga.

Esto satisface RNF-17 y respeta una distinción que el backend se tomó el trabajo de mantener: un código de “no existe” y uno de “datos inválidos” nunca se comparten, y el de conflicto de estado —arrancar una orden ya ejecutada— es un tercero distinto de los dos. **Si el frontend los funde en un “ha ocurrido un error”, tira a la basura una decisión de diseño del servidor.**

Las tres familias se presentan de forma distinta: lo que no existe explica qué referencia falló; lo inválido señala el campo; el conflicto de estado dice qué impide la operación **ahora**.

La redacción sigue el tono de voz del manual de marca (5.5): directa e instructiva, sin disculpas. “Esta orden ya se ejecutó y no admite cambios” y no “¡Ups! Algo salió mal”.

5. Sistema visual

5.1. La fuente es el manual de marca

El sistema visual no se define en este documento: lo define el *Manual de Marca de MalphasOS* v1.0, y aquí sólo se declara cómo se traduce a una interfaz.

Vive en Documentation/FrontendDesign/BrandManual/. Es un documento cerrado y anterior a esta declaración, de modo que **donde los dos discrepen, manda el manual**, salvo en lo que la sección 5.3 señala explícitamente como extensión y no como cambio.

Lo que el manual fija y esta declaración adopta sin discutir:

- **Una paleta monocroma** —tinta y grises sobre papel— con **un solo rojo de acento**, reservado a la acción principal y a las alertas, que **nunca cubre más del 10 % de una composición**.
- **Radio cero**. Ángulos rectos en todo: botones, campos, tarjetas, imágenes.
- **Alineación a la izquierda siempre**. No se centra ni se justifica.
- **Una sola familia tipográfica**, Archivo, con tres pesos.
- **Los estados operativos se distinguen por peso tipográfico y por regla, no por colores nuevos**.

Ese último punto es el que más cambia una interfaz respecto de lo habitual, y se recoge tal cual: **una orden vencida no se pinta de rojo**, se compone con otro peso. El acento señala acciones y alertas, no clasifica estados.

5.2. Color

Nombre	Hex	Papel en la interfaz
Tinta	#201E1D	Texto principal
Negro marca	#2D2B2B	Superficies oscuras, paso 900 de la escala neutra
Papel	#F3F2F2	Fondo

Nombre	Hex	Papel en la interfaz
Acento	#EC3013	Bordes, anillo de foco, iconografía de alerta, texto grande
Acento 700	#AE1800	Texto en acento y relleno de la acción principal

Más dos escalas de nueve pasos, neutra y de acento, que el manual define y de las que salen los grises intermedios y los fondos de aviso.

5.2.1. La regla de contraste del manual, y el caso que no nombra

El manual ya trae su propia regla de contraste: “El rojo #EC3013 no se usa para texto de párrafo sobre papel. Para texto en acento emplear #AE1800 (paso 700)”. Medido, funciona:

Combinación	Contraste	AA texto normal
Tinta #201E1D sobre papel	14,86:1	Cumple
Negro marca #2D2B2B sobre papel	12,60:1	Cumple
Acento 700 #AE1800 sobre papel	6,41:1	Cumple
Acento #EC3013 sobre papel	3,76:1	No cumple — por eso existe la regla

El caso que el manual no cubre es el botón. Describe la acción principal como “relleno acento, etiqueta a la izquierda”. Con el relleno en #EC3013 y la etiqueta en papel, el contraste es **3,76:1**: el control más repetido de la aplicación no alcanzaría el nivel declarado en la sección 6.

Resolución: el relleno de la acción principal usa el paso 700, #AE1800, con la etiqueta en papel. Da 6,41:1.

No es una corrección del manual sino su propia regla aplicada un caso más allá: el manual ya había decidido que el acento puro no sostiene texto encima; sólo no nombró el relleno sólido entre esos casos. El acento se queda exactamente donde el manual lo pone para todo lo demás —bordes, anillo de foco, iconografía de alerta—, y ahí **sí cumple**: para elementos no textuales el umbral es 3,0 y #EC3013 da 3,76:1.

5.3. Tipografía, retícula y espaciado

Todo lo de esta subsección viene del manual; sólo el último punto es una extensión propia, y se señala como tal.

Aspecto	Regla
Familia	Archivo. Sustitutas: Helvetica Neue, luego Arial
Pesos	Títulos 700, subtítulos 600, cuerpo 400
Interlineado	Títulos 0,95–1,05; cuerpo 1,5–1,6
Medida de lectura	Máximo 70 caracteres
Retícula	12 columnas, medianil 24 px, márgenes 48 px
Espaciado	Múltiplos de 8 px
Radio de esquina	Cero, en todo

Aspecto	Regla
Iconografía	Lucide, trazo 2 px, esquinas rectas, 16 / 20 / 24 px

Extensión propia: el manual fija retícula y márgenes pensando en composiciones de escritorio y no habla de pantallas estrechas. **En anchos de teléfono la retícula se reduce a 4 columnas y los márgenes a 16 px, conservando el medianil de 24 px y el espaciado en múltiplos de 8.** Se declara aquí porque la aplicación es móvil primero (5.4) y el manual no resuelve ese caso.

5.4. Densidad táctil y orden de diseño

- **Área táctil mínima de 44 por 44 píxeles** para todo control accionable. Es lo que convierte RNF-14 en algo medible en lugar de una intención. **Es compatible con el radio cero y con el espaciado de 8:** 44 no es múltiplo de 8, pero es una superficie mínima de interacción, no una medida de maquetación.
- **Móvil primero.** Las pantallas se diseñan para el teléfono y se expanden hacia el escritorio. Se sigue del apartado 2.1 de la ERS: el teléfono del ingeniero no es un caso degradado, es el caso principal.
- **Tamaño base de 16 píxeles** en el cuerpo. Por debajo, los navegadores móviles amplían al enfocar un campo, que es un salto visual que confunde.

5.5. Tono de los textos de la interfaz

El manual fija el tono de voz y afecta directamente a RNF-17: **directo e instructivo**, nunca entusiasta ni publicitario. Su propio ejemplo —“Orden cerrada. Próximo mantenimiento: 14 días”— es el modelo para los mensajes del sistema.

Los mensajes de error siguen ese tono: dicen qué pasó y qué hacer, sin disculpas ni signos de exclamación. La traducción de los códigos del backend (4.4) se redacta bajo esta regla.

5.6. Cómo se cuenta un paso (RNF-15)

RNF-15 pide procesos en seis pasos o menos y no define qué es un paso. Este documento lo define para poder comprobarlo:

Un paso es cada interacción que el usuario debe completar para avanzar: abrir una pantalla, rellenar un campo obligatorio, elegir de una lista o confirmar. No cuentan la navegación de vuelta, las acciones opcionales ni los datos que el sistema rellena solo.

Bajo esa definición, crear una orden de trabajo son cinco pasos: cliente, sede, áreas, equipos, y datos del servicio con la confirmación. Queda uno de margen.

6. Accesibilidad

La aplicación declara conformidad con WCAG 2.1 nivel AA, y se verifica automáticamente.

La ERS pide accesibilidad sin fijar nivel (RNF-12). Sin un nivel, el requisito no tiene criterio de aceptación y no se puede decir si se cumplió. Este documento lo fija.

Criterio	Cómo se cumple y se comprueba
Contraste $\geq 4,5:1$ en texto	Acento 700 #AE1800 para texto y para el relleno de la acción principal (5.2.1)
Área táctil ≥ 44 px	Tamaño mínimo en los componentes base
Foco siempre visible	Indicador de foco propio, nunca suprimido
Operable por teclado	Heredado de las primitivas del CDK, verificado en pruebas
Formularios etiquetados	Etiqueta asociada y error vinculado a su campo
Errores anunciados	Región de anuncio para lectores de pantalla

La verificación no es opcional. Las pruebas de integración ejecutan un analizador automático de accesibilidad sobre cada pantalla, y un fallo rompe la batería. La razón está aprendida en este mismo proyecto: **una regla declarada y no ejercitada envejece sola**, y ya ocurrió con una regla de negocio que se dio por construida durante una jornada entera sin estarlo.

Queda dicho lo que la comprobación automática no cubre: detecta una parte de los problemas, no todos. El orden de lectura, la claridad del lenguaje y la utilidad real para un lector de pantalla exigen revisión humana.

7. Movilidad y comportamiento sin conexión

7.1. Lo que se construye ahora

La aplicación se construye como una aplicación web responsiva y táctil. No es instalable y no funciona sin conexión.

Las dos capacidades están decididas y aplazadas, no descartadas: la aplicación tendrá instalación en el dispositivo y consulta sin conexión para el trabajo en campo. Se añadirán más adelante, y esta sección deja escrito el diseño con el que entrarán para que quien lo retome no vuelva a decidirlo.

7.2. Por qué aplazarlo no incumple la especificación

Conviene que quede dicho, porque el apartado 2.1 de la ERS habla de movilidad y podría leerse como que exige una aplicación instalable.

No la exige. Lo que pide es que el sistema sea accesible desde el teléfono del ingeniero **sin instalar aplicaciones nativas. Una página web responsiva abierta en el navegador del teléfono satisface ese requisito por completo**, y de hecho lo satisface de la forma más literal posible: no hay nada que instalar.

Lo mismo con el apartado 3.1.2, que lista la pantalla táctil entre las interfaces de hardware: eso lo cubre el diseño táctil de la sección 5.4, que sí entra desde el primer día.

Ningún requisito de la ERS pide instalación ni uso sin conexión. Son mejoras del escenario de campo, no obligaciones, y por eso se pueden aplazar sin dejar nada incumplido.

7.3. Lo que se añadirá después, y con qué forma

- **Instalación en el dispositivo.** Manifiesto e icono para que el ingeniero pueda anclar la aplicación a su pantalla de inicio y abrirla a pantalla completa. Sigue sin instalarse nada nativo.
- **Consulta sin conexión, de solo lectura.** Órdenes asignadas al ingeniero, con sus equipos, sedes y áreas. Cubre el escenario de campo más valioso: **llegar al hospital sabiendo qué hay que hacer.**
- **Escritura sin conexión: no,** y no por falta de tiempo. Ver 7.4.

7.4. Por qué la escritura sin conexión no entrará sin tocar el backend

Se evaluó permitir crear y avanzar órdenes sin señal, encolando los envíos, y **se descartó porque la factura sería del backend.** Lo que sigue está comprobado sobre el código, no supuesto:

- **No hay claves de idempotencia.** Una orden creada sin señal y reintentada al reconectar se crearía dos veces: el identificador lo genera hoy el servidor en cada llamada.
- **No hay bloqueo optimista.** No existe columna de versión en ninguna de las seis migraciones, de modo que dos ediciones simultáneas no se detectan.
- **Reintroduciría una mentira histórica que el diseño ya impide.** Una unidad de equipo puede trasladarse de área. Una orden creada sin conexión congelaría el área que el dispositivo tenía en caché, que puede no ser la real. El módulo de órdenes se diseñó explícitamente para que el área congelada sea siempre la verdadera, y la escritura sin conexión abriría esa puerta por detrás.

Mientras la escritura exija conexión, el backend se mantiene cerrado. Ésa es la razón de fondo por la que la consulta sin conexión es de solo lectura.

7.5. Decisiones ya tomadas para cuando se retome

Para que retomarlo sea construir y no volver a decidir:

- **Si algún día entra la escritura sin conexión,** el camino decidido es que **el cliente genere el identificador de la orden y el backend lo acepte en lugar de generarlo.** Reenviar la misma orden escribiría entonces sobre la misma fila. Es un cambio acotado —el agregado ya usa identificadores universales— pero toca dominio, aplicación y capa de servicios con sus pruebas, y **el backend dejaría de estar cerrado.**
- **Qué se guardará en el dispositivo:** únicamente lo asignado al ingeniero que ha iniciado sesión, y se borrará al cerrar sesión. Sin cifrado propio: se confía en el del dispositivo, que es lo honesto, porque una clave que vive en la misma página que descifra los datos protege menos de lo que aparenta.
- **Un supuesto que conviene no perder:** ese límite lo impondría el cliente, **no el servidor.** El backend no filtra por propietario —es deuda conocida y declarada— de modo que un usuario con permiso de lectura puede solicitar el conjunto completo. La caché respetaría un límite que nadie obliga a respetar.

8. Estrategia de pruebas

El backend llega a este punto con 624 pruebas, cero fallos y la costumbre de **verificar por mutación**: romper la producción a propósito para comprobar que algo se queja. El frontend adopta el mismo listón.

Nivel	Qué cubre
Unitarias	Validación de formularios, guards, traducción de errores, lógica de los servicios de módulo
Integración	La pantalla completa contra un API simulado, con los códigos de error reales del backend
Extremo a extremo	Inicio de sesión y creación de una orden, contra el sistema real
Accesibilidad	Analizador automático sobre cada pantalla, dentro de las pruebas de integración

Tres decisiones sobre cómo se prueba:

- **El API simulado responde con los códigos reales.** Simular un error genérico probaría que la pantalla muestra algo, no que muestra lo correcto. La prueba de que `ERR_WORK_ORDER_002` se convierte en una frase útil sólo vale si usa ese código.
- **Las pruebas de extremo a extremo son pocas y deliberadas.** Sólo cubren lo que ningún otro nivel puede: que Keycloak y el API se hablen de verdad. Es justamente el tipo de integración que compila y no funciona.
- **Una prueba que se dobla para pasar describe el defecto en vez de detectarlo.** Regla heredada del backend, donde ya evitó cerrar en falso un fallo real.

9. Plan de construcción

El primer bloque es una rebanada vertical: arranque de la aplicación, autenticación y el flujo completo de órdenes de trabajo.

Por qué vertical y no por capas. Atraviesa todo el sistema de una vez —sesión, datos, formularios, errores, accesibilidad— y cualquier decisión equivocada de este documento aparece en la primera semana y no en la quinta.

Qué arrastra consigo. El formulario de una orden obliga a construir, de camino, el selector de cliente, el de sede de ese cliente, la selección múltiple de áreas de la sede y la de equipos del área. Son piezas que otros módulos reutilizarán.

Qué cierra. Los cuatro requisitos que hoy están abiertos únicamente por falta de formulario: RF-03, RF-04, RF-06 y RF-07.

Después, por dependencias: clientes y equipos completos, y luego los módulos que esperan backend.

10. Riesgos asumidos y lo que este documento no decide

Riesgo	Por qué se acepta
Dependencia de datos marcada como experimental	Aislada tras un servicio por módulo: si cambia, se tocan los servicios y no las pantallas
Se descarta un componente reutilizable	La coherencia estructural con el backend se consideró más valiosa que ahorrar cuatro archivos
El acento no alcanza AA como relleno de botón	Medido, no estimado. Resuelto con el paso 700 del propio manual, #AE1800, que es su regla aplicada a un caso que no nombraba
La caché respetaría un límite que el servidor no impone	Declarado en 7.5. No es un riesgo vivo mientras la caché esté aplazada

Lo que este documento deja abierto a propósito: la versión exacta del framework, que se fija al crear el proyecto; el diseño de cada pantalla, que corresponde al momento de construirla; la *landing page*; los módulos cuyo backend no existe; y **cuándo entran la instalación y el uso sin conexión**, que están decididos en forma pero no en fecha.

11. Trazabilidad

Requisito	Dónde lo atiende este documento
RNF-02	3.5 y 3.3 — formularios reactivos y componentes de autocompletado
RNF-05	9 — una sola entrada de los datos generales en el flujo de la orden
RNF-07	3.5 — regla del formulario, no atributo visual
RNF-12	6 — nivel AA declarado y verificado
RNF-13	5.3 y 5.4 — retícula reducida en teléfono, y móvil primero
RNF-14	5.4 — área táctil mínima de 44 píxeles
RNF-15	5.6 — definición de paso y recuento del flujo principal
RNF-16	3.3 — primitivas accesibles de lista, autocompletado y selección múltiple
RNF-17	4.4 y 5.5 — catálogo de traducción por módulo, con el tono de voz del manual
RNF-03, 04	3.4 — caché e invalidación como vía para los umbrales de tiempo
RF-03, 04, 06, 07	9 — objetivo explícito del primer bloque
ERS 2.1	7.1 y 7.2 — web responsiva en el navegador del teléfono, sin nada que instalar
ERS 3.1.2	5.4 — diseño táctil

Este documento se revisa cuando una de sus decisiones resulte falsa. La corrección se escribe dejando constancia de qué decía antes y cuándo dejó de ser cierto, que es la convención del resto de la documentación del proyecto.